

Department of Electronic and
Telecommunication Engineering
University of Moratuwa

EN3150 - Pattern Recognition



Assignment 02: Learning from data and related
challenges and classification

210490L Prabodha K.P.K.A.

Contents

1	Logistic regression	2
1.1	Load Data	2
1.2	Explanation of <code>y_encoded = le.fit_transform(df_filtered['species'])</code>	2
1.3	Explanation of <code>X = df_filtered.drop(['species', 'island', 'sex', 'class_encoded'], axis=1)</code>	2
1.4	Why cannot use "island" and "sex" features?	3
1.5	Train a logistic regression model	3
1.6	Usage of <code>random_state=42</code>	3
1.7	Why is accuracy low? why does the saga solver perform poorly? . . .	3
1.8	Classification accuracy using <code>logreg = LogisticRegression(solver='liblinear')</code>	4
1.9	Why 'liblinear' solver perform better than 'saga' solver?	4
1.10	Compare the performance of the "liblinear" and "saga" solvers with feature scaling.	4
1.11	Run the code given in listing and solve the problem	5
1.12	Suppose you have a categorical feature with the categories 'red', 'blue', 'green', 'blue', 'green'. After encoding this feature using label encoding, you then apply a feature scaling method such as Standard Scaling or Min-Max Scaling. Is this approach correct? or not? What do you propose ?	5
1.13	Question 2	6
2	Logistic regression on real-world data	7
2.1	Choose a data set	7
2.2	Plot Correlation matrix, Pair plots using <code>sns.pairplot</code> and Comment on your results	7
2.3	Fit a logistic regression model	9
2.4	Obtain p-values and features can be discarded	10
3	Logistic regression First/Second-Order Methods	11
3.1	Generate data using code	11
3.2	Implement batch gradient descent to update the weights	11
3.3	About Loss Function	11
3.4	Loss with respect to number of iterations	12
3.5	Step 2 for stochastic Gradient descent	12
3.6	Implement Newton's method	13
3.7	Plot the loss with respect to number of iterations	14
3.8	All plots in one	14
3.9	Two approaches to decide number of iterations	15
3.10	Convergence Analysis of Batch Gradient Descent with New Centers .	16

1 Logistic regression

1.1 Load Data

1.2 Explanation of

```
y_encoded = le.fit_transform(df_filtered['species'])
```

- The dataset includes a categorical column labeled 'species' that contains names such as 'Adelie' and 'Chinstrap'.
- Machine learning algorithms, including `LogisticRegression`, want numerical input instead of categorical string labels.
- The purpose of this process is to transform the categorical **species** names into numerical labels, which then they can be used as the target variable in a machine learning model.
- The `LabelEncoder` object (denoted as `le`) is utilized to associate (map) each unique categorical value (species names) with a specific integer.
- Like this:
 - 'Adelie' - assigned the value 0
 - 'Chinstrap' - assigned the value 1
- This method integrates both `fit()` and `transform()` into a single operation, learning the mapping and promptly applying it to the dataset.
- The resulting array, `y_encoded`, consists of numerical values that represent the encoded species names, which will serve as the target variable (`y`) in the classification model.

1.3 Explanation of

```
X = df_filtered.drop(['species', 'island', 'sex', 'class_encoded'],  
axis=1)
```

- In the realm of machine learning, datasets are generally partitioned into **features** (denoted as `X`) and a **target variable** (denoted as `y`).
- The target variable is what the model is trying to predict (in this case, likely the species of penguins), and the features are the input variables that help in making this prediction.
- This line is designed to **remove** certain columns from the dataset that do not need to be used as features for the model.
- Remove the **target variable** (`class_encoded`) also.

Columns Removed : 'species', 'island', 'sex', 'class_encoded'

1.4 Why cannot use "island" and "sex" features?

- The features "island" and "sex" in the dataset are classified as categorical variables, indicating that they represent distinct categories or groups (such as various islands or gender classifications).
- Machine learning algorithms, cannot be directly incorporated into the model.
- It would be necessary to transform them into a numerical format through methods such as encoding (for instance, one-hot encoding or label encoding).
- Nevertheless, the current code opts to drop these features rather than encode them, which explains their absence as inputs for the model.

1.5 Train a logistic regression model

	coef	std err	z	P> z	[0.025	0.975]
const	-1.1569	0.184	-6.302	0.000	-1.517	-0.797
x1	0.7612	0.178	4.276	0.000	0.412	1.110
x2	-0.8385	0.201	-4.173	0.000	-1.232	-0.445
x3	0.7176	0.174	4.130	0.000	0.377	1.058
x4	-0.1326	0.165	-0.803	0.422	-0.456	0.191
x5	0.1531	0.169	0.907	0.364	-0.178	0.484

Figure 1: Enter Caption

1.6 Usage of random_state=42

- The `random_state` parameter does the shuffling task of data before splitting by setting a *seed* for the random number generator.
- Setting `random_state=42` ensures that the random splitting process is *consistent* every time the code is run, meaning the training and testing data will always be split the same way, this guaranteeing the reproducibility of results.

1.7 Why is accuracy low? why does the saga solver perform poorly?

Accuracy: 0.5813953488372093

- The accuracy is relatively low, approximately 0.58, due to the `saga solver` being unsuitable for this dataset. This solver is designed for large datasets.
- This dataset comprises only **240** data points, which is considered small.
- The application of the saga solver in this situation may lead to a slower and more complex process, mostly not ending with an optimal solution.

- Moreover, the saga solver exhibits sensitivity to the scaling of features. If the data is not correctly scaled, it may lead to diminished accuracy.
- For these reasons, the saga solver is likely to **underperform** in this scenario.

1.8 Classification accuracy using

```
logreg = LogisticRegression(solver='liblinear')
```

Accuracy is **1.0**

1.9 Why 'liblinear' solver perform better than 'saga' solver?

- The "liblinear" solver has more performance compared to the "saga" solver in this binary classification scenario, where only two output classes are used.
- This solver is designed for these tasks, so it doesn't need a lot of hyperparameter tuning to work well.
- Liblinear is not impacted by smaller datasets, unlike the saga solver, which may face difficulties in such scenarios. It is less sensitive to feature scaling, allowing it to produce satisfactory results even when the data is not scaled.
- liblinear is more effective in this context.

1.10 Compare the performance of the "liblinear" and "saga" solvers with feature scaling.

Solver	Accuracy before Scaling	Accuracy after Scaling
Saga	0.58	0.98
Liblinear	1	0.98

Table 1: Accuracy before and after Scaling

- The table shows that the accuracy of the saga solver is enhanced after scaling. This improvement occurs because the saga solver is quite responsive to feature scaling. When the features are scaled, their performance sees a notable boost.
- Conversely, the liblinear solver's accuracy remains relatively the same with scaling, as it is not performed more by feature scaling. The minor decrease in accuracy may be attributed to variations in feature magnitude post-scaling.
- In summary, liblinear works better than saga before scaling, but after scaling, both solvers have nearly the same accuracy.

1.11 Run the code given in listing and solve the problem

- Issues in Listing 3: When running this code, the problem exists in the dataset that contains categorical features. The attempt to train a model has been made without their due encoding. The main problem is that the model is not capable of handling categorical data which hasn't been encoded.
- This can be fixed by encoding these categorical features before the training of the model. The solution can be one-hot encoding of such features as "island" and "sex", which then finally turns into a numerical format for the model to train.

1.12 Suppose you have a categorical feature with the categories 'red', 'blue', 'green', 'blue', 'green'. After encoding this feature using label encoding, you then apply a feature scaling method such as Standard Scaling or Min-Max Scaling. Is this approach correct? or not? What do you propose ?

- This is not the right approach.
- Label encoding can create a false sense of order between categories because it assigns numbers to categories. For instance, using Label encoding might imply that "green" is greater than "red", whereas, in reality, the categories don't have any numerical relationship with each other.
- If any scaling method is used, like Standard Scaling or Min-Max Scaling, after performing label encoding, then these categories are treated as if they were continuous values.
- This is inappropriate for categorical data. Instead, one-hot encoding can replace label encoding. One-hot encoding does not introduce any ordinal relationships among the categories.
- Once the one-hot encoding is done, scaling is not necessary on these categorical features; however, scaling can be still done on any numerical features.

1.13 Question 2

1.

$$y = w_1x_1 + w_2x_2 + w_0$$

$$y = 0.06 \cdot 50 + 1.5 \cdot 3.6 - 5.9$$

$$y = 2.5$$

$$Pr(A^+) = \frac{1}{1 + e^{-2.5}}$$

$$Pr(A^+) = 0.9241 = 92.41\%$$

2.

$$Pr(A^+) = 0.6$$

$$0.6 = \frac{1}{1 + e^{-y}}$$

$$y = 0.41$$

$$0.41 = 0.06x_1 + 1.5 \cdot 3.6 - 5.9$$

$$x = 15.167$$

2 Logistic regression on real-world data

2.1 Choose a data set

Dataset Name: Heart Failure Clinical Records

- This dataset consists of medical records from 299 patients diagnosed with heart failure, featuring 13 clinical attributes. The primary goal is to predict patient survival outcomes based on these attributes. It includes both binary and continuous features, such as age, serum creatinine, and ejection fraction, facilitating tasks like classification and regression in healthcare analytics.

2.2 Plot Correlation matrix, Pair plots using sns.pairplot and Comment on your results

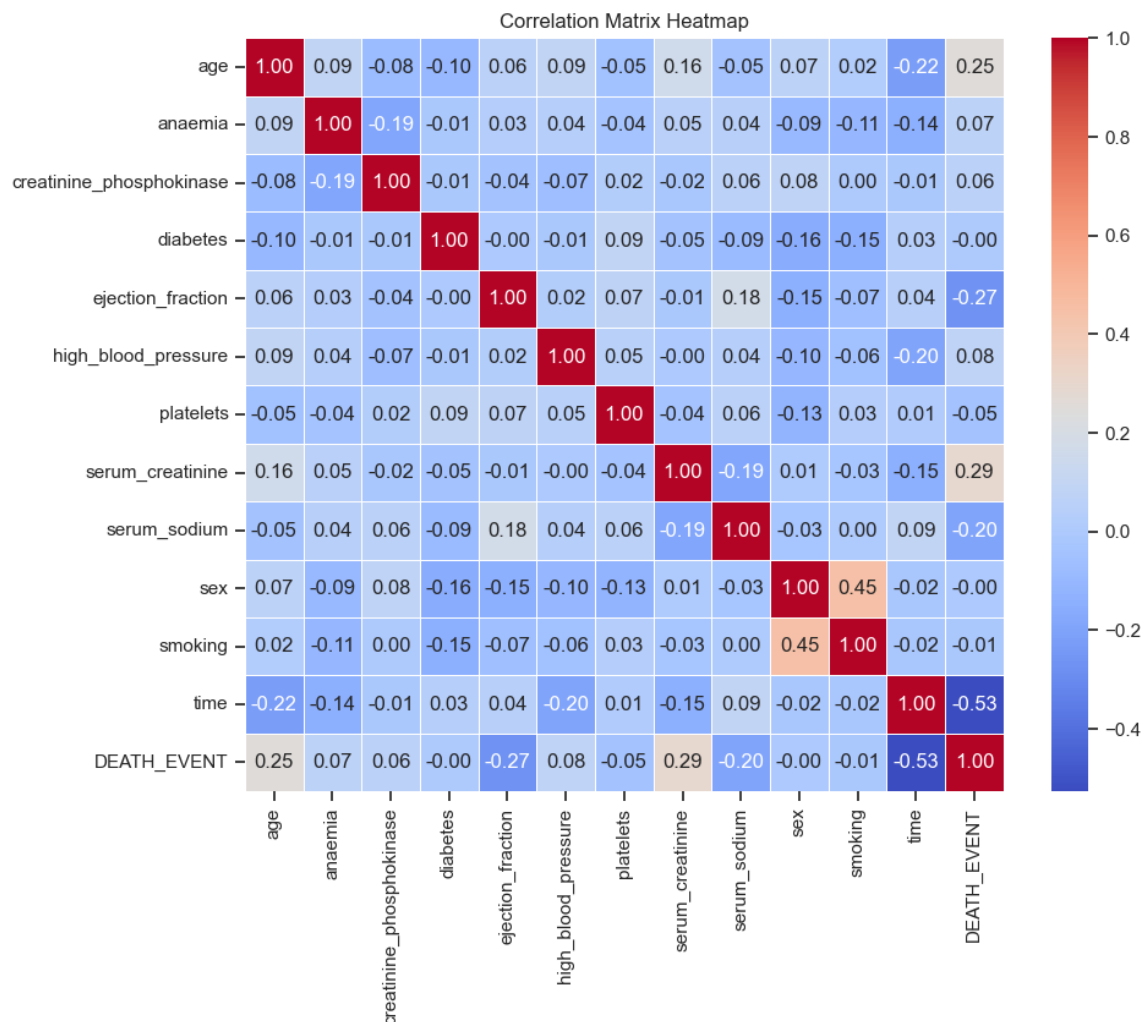


Figure 2: Correlation Matrix for all features

Selected 5 features:

- time
- ejection_fraction
- serum_creatinine
- age
- serum_sodium

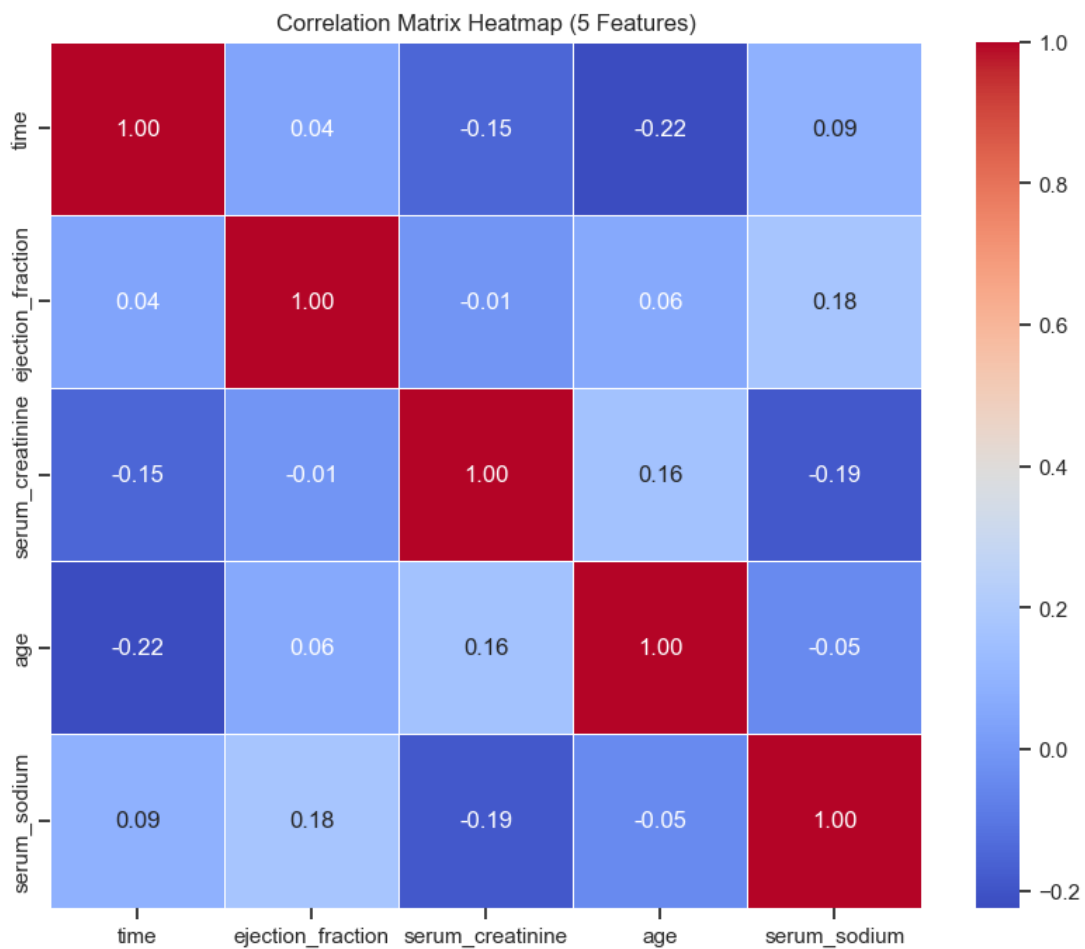


Figure 3: Correlation Matrix for 5 features

How selected features:

- select features that haven't high correlation selected.

Pair Plots

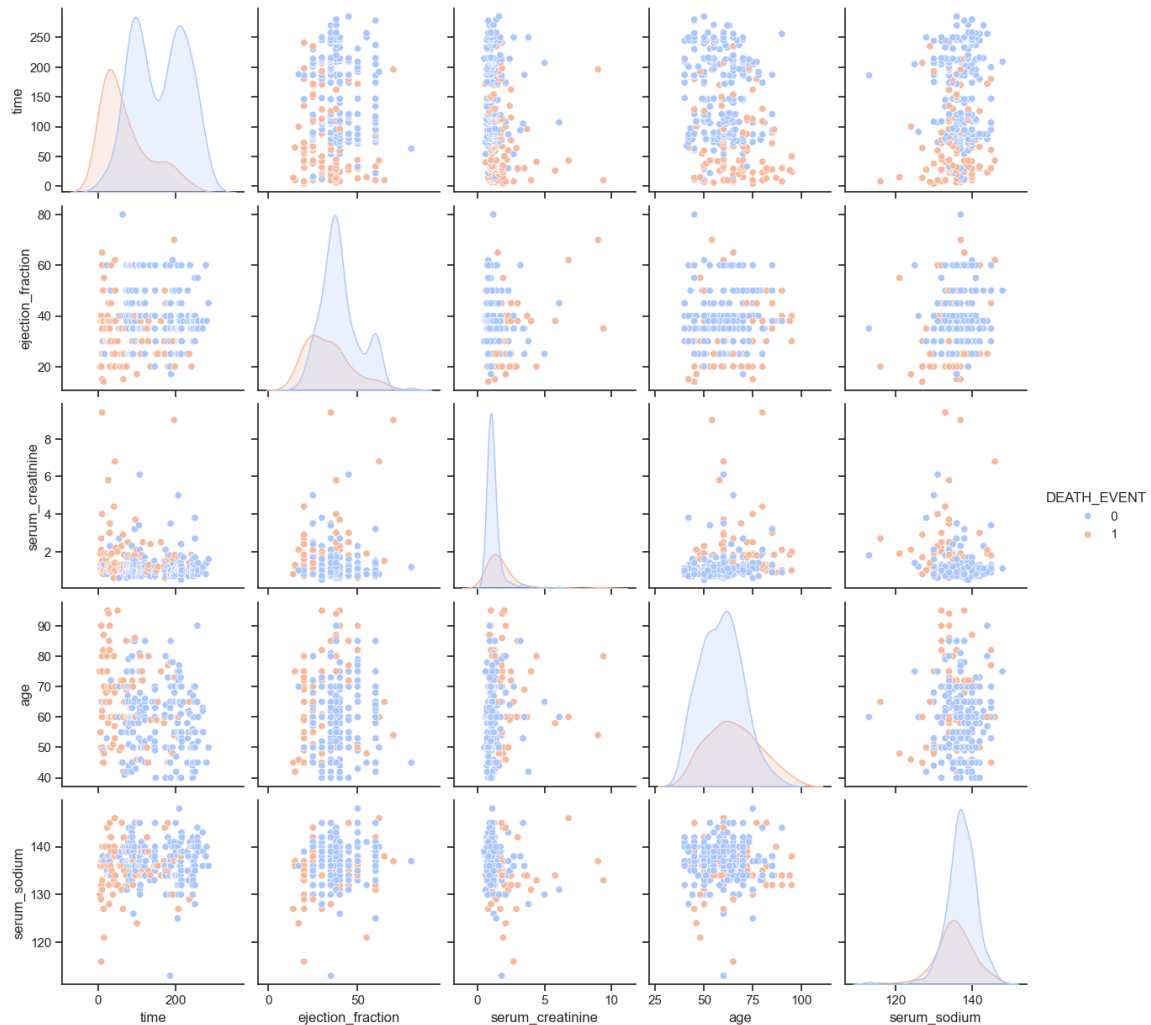


Figure 4: Pair plots

2.3 Fit a logistic regression model

Steps done in a code:

- Step 1: Split the Data:** The dataset is split into training and testing sets using `train_test_split`, with 80% for training and 20% for testing.
- Step 2: Scale the Data:** Features in the training and testing sets are standardized using `StandardScaler` to ensure the data has zero mean and unit variance.
- Step 3: Initialize Logistic Regression Model:** A logistic regression model is initialized using the `lbfgs` solver.
- Step 4: Fit the Model:** The logistic regression model is trained on the scaled training data using the `fit()` method.

Step 5: Evaluate the Model: The model is evaluated on the testing set by predicting the labels with `predict()`, calculating the accuracy using `accuracy_score()`, and printing a detailed classification report with `classification_report()`.

```

1 X = data[['age', 'ejection_fraction', 'serum_creatinine', '
    serum_sodium', 'diabetes']]
2 y = data['DEATH_EVENT'] # Dependent variable
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.2, random_state=42)
4
5 scaler = StandardScaler()
6 X_train_scaled = scaler.fit_transform(X_train)
7 X_test_scaled = scaler.transform(X_test)
8
9 log_reg = LogisticRegression()
10 log_reg.fit(X_train_scaled, y_train)
11
12 y_pred = log_reg.predict(X_test_scaled)
13
14 accuracy = accuracy_score(y_test, y_pred)
15 report = classification_report(y_test, y_pred)
16 print(f"Accuracy of the model: {accuracy * 100:.2f}%")
17 print("Classification Report:")
18 print(report)

```

Accuracy of the model: 65.00%					
Classification Report:					
	precision	recall	f1-score	support	
0	0.64	0.91	0.75	35	
1	0.70	0.28	0.40	25	
accuracy			0.65	60	
macro avg	0.67	0.60	0.58	60	
weighted avg	0.67	0.65	0.61	60	

Figure 5: Accuracy

The accuracy, precision, recall, and F1-score for class 0 are reasonable, with high recall, indicating better performance for this class. However, the model struggles with class 1, showing low recall and F1-score, indicating bias towards predicting class 0.

2.4 Obtain p-values and features can be discarded

```

1 # Fit the logistic regression model using statsmodels
2 logit_model = sm.Logit(y_train, sm.add_constant(X_train_scaled))
3 result = logit_model.fit()
4
5 # Print the summary of the model
6 print(result.summary())

```

	Coef.	Std.Err.	z	P> z	[0.025	0.975]
const	-1.1569	0.1836	-6.3015	0.0000	-1.5168	-0.7971
age	0.7612	0.1780	4.2760	0.0000	0.4123	1.1101
ejection_fraction	-0.8385	0.2009	-4.1735	0.0000	-1.2323	-0.4447
serum_creatinine	0.7176	0.1738	4.1300	0.0000	0.3771	1.0582
serum_sodium	-0.1326	0.1651	-0.8031	0.4219	-0.4562	0.1910
diabetes	0.1531	0.1687	0.9072	0.3643	-0.1776	0.4838

Figure 6: P-values of coefficient

- Considering these p-values, we can observe that all features other than **serum_sodium** and **diabetes** have a p-value of 0.
- The p-values of **serum_sodium** and **diabetes** are 0.4219 and 0.3643, respectively, which are greater than 0.05.

3 Logistic regression First/Second-Order Methods

3.1 Generate data using code

3.2 Implement batch gradient descent to update the weights

```

1 def batch_gradient_descent(X, y, weights, learning_rate, iterations)
  :
2     m = X.shape[0]
3     losses = []
4     for i in range(iterations):
5         predictions = sigmoid(X.dot(weights))
6         gradient = (1/m) * X.T.dot(predictions - y)
7         weights -= learning_rate * gradient
8         loss = compute_loss(X, y, weights)
9         losses.append(loss)
10    return weights, losses

```

The graph shows loss plotted against the number of iterations. The loss decreases with an increase in the number of iterations. The loss decreases moderately; it is being reduced smoothly because batch gradient descent uses all the dataset.

3.3 About Loss Function

Binary Cross Entropy was used.

Reasons:

- It punishes more for very confident wrong predictions.
- It ensures a smooth optimization process.
- It can work with probabilistic outputs.

Formula:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

3.4 Loss with respect to number of iterations

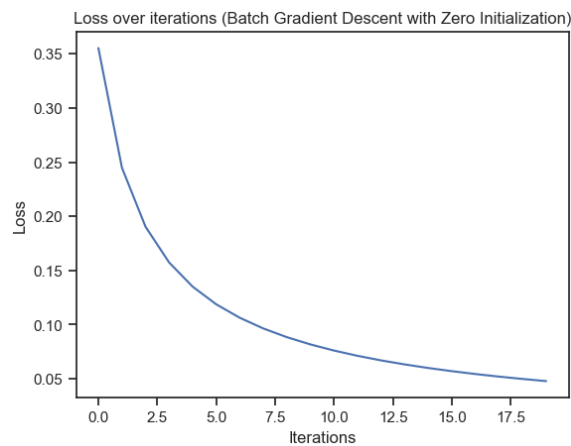


Figure 7: Lost function in gradient descent

3.5 Step 2 for stochastic Gradient descent

```
1  def stochastic_gradient_descent(X, y, weights, learning_rate,
2  iterations):
3      m = len(y)
4      losses = []
5      for i in range(iterations):
6          rand_index = np.random.randint(0, m)
7          xi = X[rand_index].reshape(1, -1)
8          yi = y[rand_index].reshape(-1)
9
10         prediction = sigmoid(xi.dot(weights))
11
12         gradient = xi.T.dot(prediction - yi)
13         weights = weights - learning_rate * gradient
14
15         loss = compute_loss(X, y, weights)
16         losses.append(loss)
17     return weights, losses
```

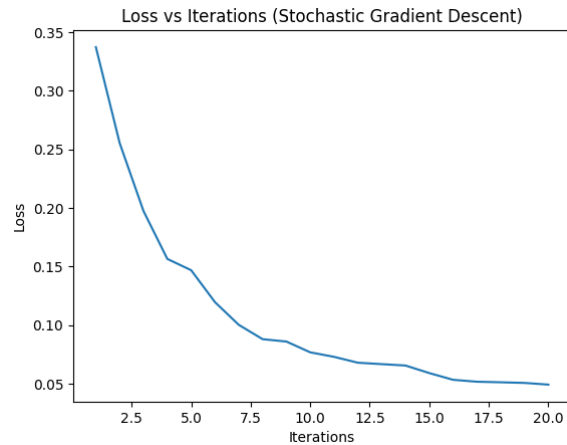


Figure 8: Lost function in stochastic gradient descent

3.6 Implement Newton's method

Newton's method makes use of a second-order derivative, the Hessian matrix, to update the weights more precisely for quicker convergence. Here's how to do that:

```
1 def newtons_method(features, labels, initial_weights, iterations):
2     m = features.shape[0]
3     losses = []
4     weights = initial_weights
5     for i in range(iterations):
6         predictions = sigmoid(features.dot(weights))
7         gradient = (1/m) * features.T.dot(predictions - labels)
8         H = (1/m) * features.T.dot(np.diag(predictions * (1 -
9 predictions))).dot(features)
10        weights -= np.linalg.inv(H).dot(gradient)
11        loss = compute_loss(features, labels, weights)
12        losses.append(loss)
13    return weights, losses
```

3.7 Plot the loss with respect to number of iterations

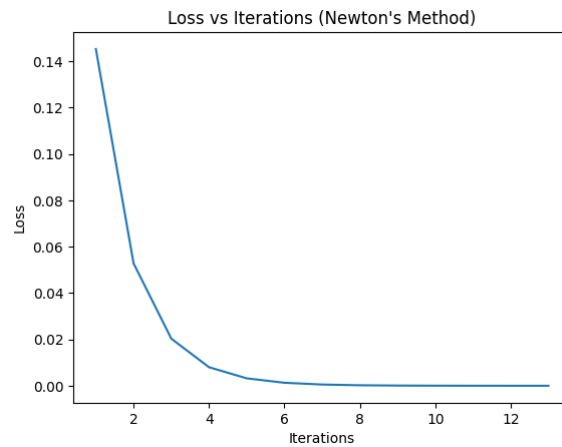


Figure 9: Lost function using Newton's Method

3.8 All plots in one

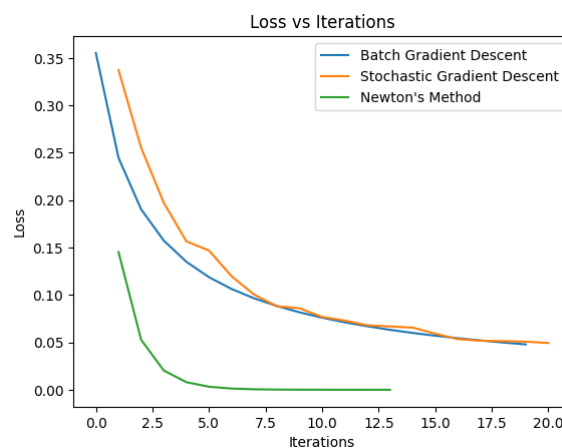


Figure 10: All in one

The graph compares the loss function for three different optimization methods: Batch Gradient Descent, Stochastic Gradient Descent, and Newton's Method.

Batch Gradient Descent

Batch Gradient Descent is a method of finding the gradient of the loss function by considering all data points in the dataset.

- It converges very slowly but steadily.
- Often used in the case of small datasets.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) calculates the gradient of the loss function using a single data point at each iteration.

- Convergence is much faster compared to Batch Gradient Descent.
- However, it introduces more noise and may not always converge to the global minimum of the cost function.

Newton's Method

Newton's Method performs second-order optimization based on the Hessian matrix to calculate the step size.

- It converges very quickly, typically reaching a very low loss value in only a few iterations.
- The downside is that it requires high computational resources.

Key Observations

- Newton's Method converged the fastest to a very low loss value.
- Batch Gradient Descent converged slowly but steadily.
- Stochastic Gradient Descent converged faster than Batch Gradient Descent but introduced noise, and it may not always converge to the global minimum.

Conclusion

Which optimization method to use depends on several factors:

- **Dataset size:** For small datasets, Batch Gradient Descent is suitable.
- **Computation resources:** For larger datasets, Stochastic Gradient Descent or Newton's Method can be more efficient.
- **Desired convergence:** Newton's Method converges quickly but is computationally expensive, while Stochastic Gradient Descent is faster but less stable in its convergence.

3.9 Two approaches to decide number of iterations

- **Cross-Validation:** Use cross-validation to determine the optimal number of iterations. The model should be trained for different iteration counts, performances on validation data should be evaluated, and the number of iterations that leads to the best generalization performance should be chosen.

- **Early Stopping:** Create early stopping that would be based on the tracking of validation loss during training. If the validation loss doesn't improve in a number of consecutive iterations, then training should stop.

3.10 Convergence Analysis of Batch Gradient Descent with New Centers

```
1 # Generate synthetic data for the first set of centers
2 np.random.seed(0)
3 centers_first = [[-5, 0], [5, 1.5]]
4 X_first, y_first = make_blobs(n_samples=2000, centers=centers_first,
5                               random_state=5)
6 transformation_first = [[0.5, 0.5], [-0.5, 1.5]]
7 X_first = np.dot(X_first, transformation_first)
8
9 # Generate synthetic data for the second set of centers
10 np.random.seed(0)
11 centers_second = [[3, 0], [5, 1.5]]
12 X_second, y_second = make_blobs(n_samples=2000, centers=
13                                 centers_second, random_state=5)
14 transformation_second = [[0.5, 0.5], [-0.5, 1.5]]
15 X_second = np.dot(X_second, transformation_second)
16
17 # Initialize weights for the first dataset
18 weights_first = np.random.randn(X_first.shape[1]) * 0.01
19 weights_first, losses_first = batch_gradient_descent(X_first,
20                                                       y_first, weights_first, learning_rate=0.1, iterations=20)
21
22 # Initialize weights for the second dataset
23 weights_second = np.random.randn(X_second.shape[1]) * 0.01
24 weights_second, losses_second = batch_gradient_descent(X_second,
25                                                         y_second, weights_second, learning_rate=0.1, iterations=20)
26
27 # Plot the loss over iterations for both datasets
28 plt.plot(range(1, 21), losses_first, label="Batch Gradient Descent (
29           First Centers)")
30 plt.plot(range(1, 21), losses_second, label="Batch Gradient Descent
31           (Second Centers)")
32 plt.xlabel('Iterations')
33 plt.ylabel('Loss')
34 plt.title('Convergence with Different Data')
35 plt.legend()
36 plt.show()
```

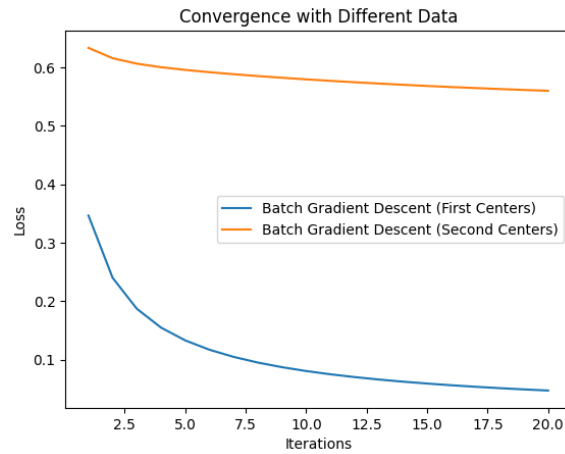


Figure 11: Convergence Comparison

Changing the centers to $[[3, 0], [5, 1.5]]$, the convergence behavior of gradient descent depends on the nature of separability that the new centroids provide to the data. The better it makes the classes linearly separable, the faster convergence is expected; otherwise, poor separability will result in more iterations for convergence. The initial centers are at $(-5, 0)$ and $(5, 1.5)$. The loss function decreases steadily over the iterations, meaning that it converges towards a good solution. Whereas Batch Gradient Descent with previous centers converges to a lower loss value, Batch Gradient Descent with new centers indicates that the previous centers were a better starting point for the algorithm.